

OpenTelemetry

Instrumentering

En kort introduksjon for utvidet standup



Instrumentering

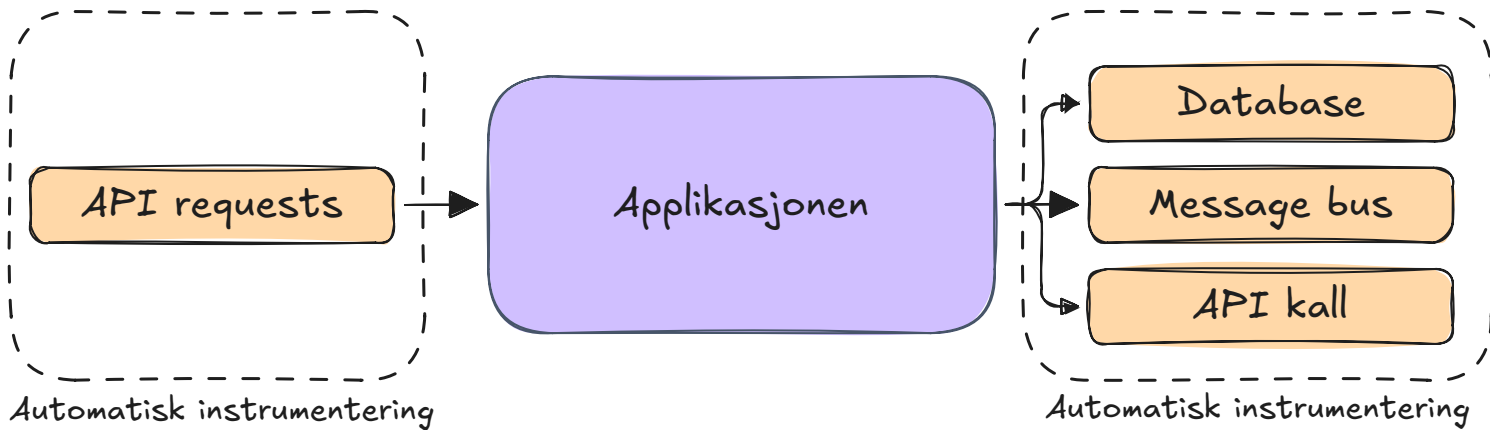
Instrumentering er det vi kaller det å fortelle koden vår at den skal sende signaler til OpenTelemetry

Det er to typer instrumentering:

- Zero-code/automatic instrumentering
- Code-based/manual instrumentering

Automatisk instrumentering

Automatisk instrumentering er en veldig rask måte å komme igang, setter opp signaler for ressurser og biblioteker som f.eks. database, requests, osv.



Automatisk instrumentering

Gir oss et innblikk i hvordan applikasjonen interagerer med omverdenen. Kan være nyttig, men er også ofte generisk informasjon som krever at du graver for å få svar.

GET /api/scheduled_trx

SELECT

WRITE

SELECT

GET <http://github.com/users/...>

MQ Push ...

Automatisk instrumentering

Det er mange populære biblioteker som har en modul for automatisk instrumentering tilgjengelig i pip.

<https://opentelemetry.io/ecosystem/registry/>

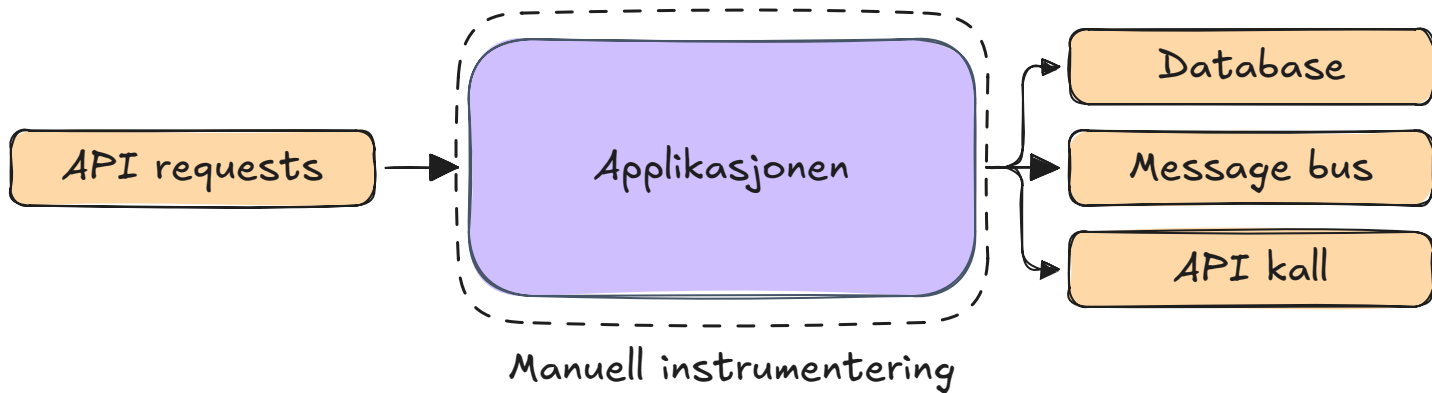
```
from opentelemetry.instrumentation.django import DjangoInstrumentor
from opentelemetry.instrumentation.psycopg2 import Psycopg2Instrumentor
from opentelemetry.instrumentation.requests import RequestsInstrumentor
```

```
# OpenTelemetry setup
```

```
RequestsInstrumentor().instrument()
Psycopg2Instrumentor().instrument()
DjangoInstrumentor().instrument()
```

Manuell instrumentering

I motsetning til automatisk instrumentering er manuell instrumentering den man selv legger inn som overvåker business-logikken til applikasjonen din



Manuell instrumentering

Når man selv lager signaler så er det en selv som bestemmer hva det er nyttig å

POST /api/user

Fetch existing users



Get metadata from GH



Store metadata



Traces og spans

Traces brukes for å få innsikt i hva applikasjonen gjør og hvordan dette passer inn i koden.

I tillegg til strukturell metadata, så kan man putte mer informasjon i dem:

- Attributes
- Events
- Span links
- Span status og kind

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

def get_metadata_from_github(user: User) -> GithubUserMetadata:

    # Creating a new span
    with tracer.start_as_current_span(
        "Get metadata from GH",
        attributes={
            "user.id": user.id,
            "user.name": user.name,
        },
    ) as span:
        client = ...
        response = client.get(...)

        # Adding an event to the span
        span.add_event(
            "Received response from GH",
            attributes={
                "request.status_code": response.status_code,
            },
        )

        ...

@tracer.start_as_current_span("Get metadata from GH")
def get_metadata_from_github(user: User) -> GithubUserMetadata: ...
```


Traces og spans

Det er ofte bedre å legge til mer informasjon i en eksisterende span enn å lage en ny.

Traces er ikke et profileringsverktøy selv om de kan se ut som flamegraphs.

God telemetri får man ved å være intensjonell med hvordan man velger signalene sine. Det tar tid.

```
import fastapi
from opentelemetry import trace
from opentelemetry.sdk.trace import StatusCode

app = fastapi.FastAPI()

@app.post("/api/user")
def create_user(user_request: CreateUserRequest) -> User:

    # Get the current span created by auto instrumentation
    span = trace.get_current_span()

    # Add attributes to existing span
    span.set_attribute("user.name", user_request.name)
    span.set_attribute("user.email", user_request.email)

    existing_user = get_user_by_email(user_request.email)
    if existing_user is not None:
        span.add_event(
            "User already exists in DB",
            attributes={
                "user.id": existing_user.id,
                "user.name": existing_user.name,
                "user.email": existing_user.email,
            },
        )

        span.set_status(StatusCode.ERROR)
        return fastapi.Response(
            content="User with that email already exists",
            status_code=400,
        )

    github_metadata = get_metadata_from_github(user_request.email)

    span.set_attribute("user.github_username", github_metadata.username)

    new_user = create_user(user_request, github_metadata)

    span.set_attribute("user.id", new_user.id)

    return new_user
```

Metrics

Metrikker defineres ved hjelp av målere, det finnes flere typer for ulike formål

- Counter
- Histogram
- Gauge
- UpDownCounter

```
# In e.g. app telemetry
import asyncio
import time

from opentelemetry import metrics

meter = metrics.get_meter("application.name")

db_watcher_tick_counter = meter.create_counter(
    name="executor.db_watcher_tick.counter",
    description="Number of db_watcher_tick() iterations executed",
    unit="1",
)

db_watcher_execution_timer = meter.create_histogram(
    name="executor.db_watcher_tick.execution_time",
    description="Measures the duration of a db_watcher_tick()",
    unit="ms",
)

# In code
class DbWatcher:
    async def tick(self):
        db_watcher_tick_counter.add(1)

        start = time.perf_counter_ns()
        new_sessions = await session_repo.get(...)
        duration = (time.perf_counter_ns() - start) * 1e6

        db_watcher_execution_timer.record(duration)

    async def run(self):
        while True:
            await self.tick()
            await asyncio.sleep(5)
```

Metric attributter

Man kan sette attributter på en meter, men vær oppmerksom på at hver unike kombinasjon av attributtverdier gir en egen dataserie. ⚠

Attributtene må derfor ha lav kardinalitet.

```
import time
from typing import Awaitable, Callable

import fastapi
from opentelemetry import metrics
from opentelemetry.semconv.attributes.http_attributes import (
    HTTP_REQUEST_METHOD,
    HTTP_RESPONSE_STATUS_CODE,
    HTTP_ROUTE,
)

app = fastapi.FastAPI()
meter = metrics.get_meter("application.name")

http_request_count = meter.create_counter("request count")
http_request_duration = meter.create_histogram("request duration")

@app.middleware("http")
async def opentelemetry_metrics_middleware(
    request: fastapi.Request,
    call_next: Callable[[fastapi.Request], Awaitable[fastapi.Response]],
) -> fastapi.Response:

    start_ns = time.perf_counter_ns()

    response = await call_next(request)

    duration_ns = time.perf_counter_ns() - start_ns

    attributes = {
        HTTP_ROUTE: request.url.path,
        HTTP_REQUEST_METHOD: request.method,
        HTTP_RESPONSE_STATUS_CODE: response.status_code,
    }

    http_request_count.add(1, attributes)
    http_request_duration.record(duration_ns, attributes)

    return response
```

Attributtnavn

Konsekvent navngiving av attributter på tvers av applikasjoner er viktig for å få god telemetri.

OpenTelemetry har et bibliotek med semantiske konvensjoner som burde brukes der mulig

<https://opentelemetry.io/docs/specs/semconv/>

```
# In e.g. app.config.opentelemetry

USER_ID = "user.id"
USER_NAME = "user.name"
USER_EMAIL = "user.email"

# In main application

from app.config.opentelemetry import (
    USER_ID,
    USER_NAME,
)
from opentelemetry.semconv.attributes.http_attributes import (
    HTTP_RESPONSE_STATUS_CODE,
)

def get_metadata_from_github(user: User) -> GithubUserMetadata:

    with tracer.start_as_current_span(
        "Get metadata from GH",
        attributes={
            USER_ID: user.id,
            USER_NAME: user.name,
        },
    ) as span:
        client = ...
        response = client.get(...)

        span.add_event(
            "Received response from GH",
            attributes={
                HTTP_RESPONSE_STATUS_CODE: response.status_code,
            },
        )

    ...
```